

FROGGER GAME

JavaFX Game Project Report

Course: Object-Oriented Programming with Java Submitted

to: Prof Abdul Karim Shahid

Group Members:

Aiman Fatima (008)

Irza Hashim (152)

Ayesha Afzal (177)

Eman (140)

Date: 15 May, 2026

1.Introduction

Frogger is a classic arcade game rebuilt as a JavaFX desktop application. In this game, the player controls a frog that must cross a busy road and a dangerous river to reach safe spots on the other side of the screen. The project demonstrates core Java concepts such as object-oriented programming, game loops, keyboard input handling, animations, and 2D graphics using JavaFX.

The goal of the game is to guide the frog from the bottom of the screen to the top without getting hit by cars or falling into the river. The frog must jump on moving logs to cross the water safely. Filling all five safe spots wins the game.

Key features of the game include:

- Three difficulty levels: Easy, Medium, and Hard
- Smooth movement of cars and logs
- Collision detection for roads and rivers
- Night mode and animated water effects
- Particle and death animations
- Sound effects and screen shake effects
- Resizable game window
- Menu screen, score system, HUD bar, and game-over screen

2. Objectives

To develop a playable Frogger game using Java and JavaFX.

- To apply object-oriented programming concepts in a real-world project.
- To implement collision detection, animation effects, and keyboard event handling.
- To understand real-time game loops and JavaFX graphics rendering.
- To create a scalable and interactive desktop game with multiple difficulty levels.
- To enhance the game using visual effects such as particles, screen shake, and night mode.

3. System Requirements Hardware

Requirements

- Intel Core i3 or higher
- 4 GB RAM or more
- Keyboard and display monitor

Software Requirements

- Java JDK 17 or later
- JavaFX SDK
- IntelliJ IDEA / VS Code / Eclipse
- Windows 10 or later

4. How the Game Works

Overall Flow

The game runs on a loop that fires roughly 60 times every second using JavaFX's `AnimationTimer`. Each tick of this loop updates the positions of game objects, processes collisions and animations, and redraws the canvas to display the latest frame.

The game has three main screens:

- Menu: The player selects a difficulty level and presses Enter or Space to start the game.
- Playing: The main gameplay screen where the frog can move and interact with objects.
- Over: Displayed when the player wins the game or loses all lives.

World Layout

The game canvas is 480 pixels wide and 560 pixels tall, including a 40-pixel HUD bar at the bottom. The screen is divided into horizontal strips, each 40 pixels tall. From top to bottom, the zones are:

- (Y = 0 to 40) Goal line containing safe spots the frog must reach.
- (Y = 40 to 240) River zone containing moving logs and animated water effects.
- (Y = 240 to 440) Road zone containing multiple rows of moving cars.
- (Y = 440 to 520) Starting grass area where the frog begins.
- (Y = 520 to 560) HUD bar displaying score, lives, difficulty, and high score.

Movement and Collision

Every frame, each car and log moves by adding its speed value to its horizontal position. When an object goes off one side of the screen, it instantly reappears on the opposite side so the lanes always appear full and continuously moving.

The collision rules are:

- River zone not on a log: if the frog is over the river but not standing on any log, it falls into the water and loses a life.
- River zone on a log: the frog moves together with the log each frame. If the log carries the frog outside the screen boundary, a life is lost.
- Road zone: if any car overlaps with the frog, a collision occurs and a life is lost.
- Top row: reaching a new safe slot on the goal line adds it to the safe-spot list and awards the player 100 points.

- Collision effects: when the frog dies, the game triggers particle effects, screen shake, and a spinning death animation before respawning the frog.

Difficulty Levels

Three helper methods return different values depending on the chosen difficulty. The table below shows how each setting changes:

Setting	Easy	Medium	Hard
Car speed	1.5 px / frame	2.5 px / frame	3.8 px / frame
Log speed	1.2 px / frame	1.8 px / frame	2.8 px / frame
Log width	110 px (wide)	85 px	62 px (narrow)
Starting lives	5	3	1

5. Important Modules and How They Work

- **Car (Inner Class)**

The Car class stores the position, speed, width, and colour of one vehicle. A negative speed value means the car travels to the left, while a positive value means it moves to the right. Every frame, the update method adds the speed to the x position. If the car moves past the edge of the screen, it wraps around to the opposite side so traffic appears continuous. The hits method checks whether the car's rectangle overlaps with the frog's rectangle.

- **Log (Inner Class)**

The Log class is implemented similarly to the Car class in terms of movement and screen wrapping. However, its interaction with the frog differs. Instead of causing a collision, the on Log method determines whether the frog is positioned on top of a log. If the condition is true, the frog moves along with the log at the same speed. The vertical

collision boundaries are defined more precisely compared to cars to ensure accurate and challenging log-riding mechanics.

- **Start Method (Game Initialization)**

The start (Stage stage) method is the entry point of the JavaFX application. It creates the Canvas and Scene, sets up the Graphics Context, and attaches keyboard input handling to the Scene. A Scale transform is applied to support window resizing without changing game logic. The AnimationTimer is started in this method, which begins the main game loop. This method is responsible for initializing and launching the entire game.

- **Game Loop (Animation Timer)**

The game is built around JavaFX's AnimationTimer, which functions as the core game loop. The handle() method is executed continuously at approximately 60 frames per second. During each frame, the program determines the current game state.

When the game is in the Playing state, the update() method is executed to process all game logic before rendering. When the game is in the Menu or GAME OVER state, only the rendering method is executed so that the user interface remains visible and responsive. This design ensures that gameplay logic is only processed when required, improving efficiency.

- **Update Method**

- The update() method runs once per frame during gameplay and handles all core game logic. It first manages the death system, including death animation, particle effects, life reduction, and resetting or ending the game when lives reach zero. A short screen shake effect is also applied when the frog dies. Next, it updates all moving objects, including cars on the road and logs in the river, to create continuous movement in the game environment. The method then processes river logic, where the frog must stay on a log to avoid drowning. If no log is found, the frog dies. It also handles road collisions, where contact with any car results in death. Finally, it updates environmental effects such as **water ripples and particles**, ensuring smooth visual animation throughout gameplay.

- **Handle Key Method**

The handle Key method is linked to the Scene, so it runs whenever the player presses a key. It looks at the current screen state and acts accordingly:

- On the Menu screen, the Up and Down arrow keys change the selected difficulty and Enter or Space starts the game.
- On the Playing screen, the arrow keys move the frog one tile (40 pixels) in that direction. The R key goes back to the menu.
- On the Over screen, only R works, returning the player to the menu.

After each move the frog position is clamped so it can never go outside the canvas boundary.

- **Draw Method**

The draw method repaints the entire canvas from scratch every frame. Shapes drawn later appear on top of shapes drawn earlier, so the order matters. The steps are:

- Draws the **background**, including grass, river, and road areas.
- Draws **lane divider lines** on the road for visual structure.
- Draws the **safe spots (goal slots)** at the top, showing which ones are filled or empty.
- Draws all **logs** as brown rounded rectangles in the river.
- Draws all **cars** with their body colours and wheel details.
- Draws the **frog** using layered shapes for the body, eyes, and legs.
- Draws the **HUD bar**, which displays score, difficulty level, and remaining lives.
- If the game is over, it draws a **dark overlay screen** with the result message.

- **Setup Level Method**

This method runs once at the start of each game to initialize the game environment. It clears any existing cars and logs, then creates new objects for the level. The positions, speeds, and sizes of cars and logs are defined using hardcoded data arrays, where each entry represents the starting xposition, y-position, movement speed, and width of an object. Car colours are assigned in a repeating pattern from a predefined set of shades to improve visual variety. Log widths are

adjusted based on the selected difficulty level, where Easy mode provides wider logs for easier navigation, while Hard mode uses narrower logs to increase the challenge.

- **Window Scaling**

A Scale transform is applied to the canvas to support dynamic window resizing. This allows the game to visually adjust to different window sizes while keeping the internal coordinate system unchanged. Listeners are attached to the Scene's width and height properties. When the window is resized, the scale factor is recalculated automatically based on the new dimensions. Only the visual output is scaled; the game logic continues to operate in a fixed coordinate space of 480×560 pixels. This ensures consistent gameplay behaviour regardless of window size, without requiring any changes to the underlying game mechanics.

- **Sound Module (Play Beep Method)**

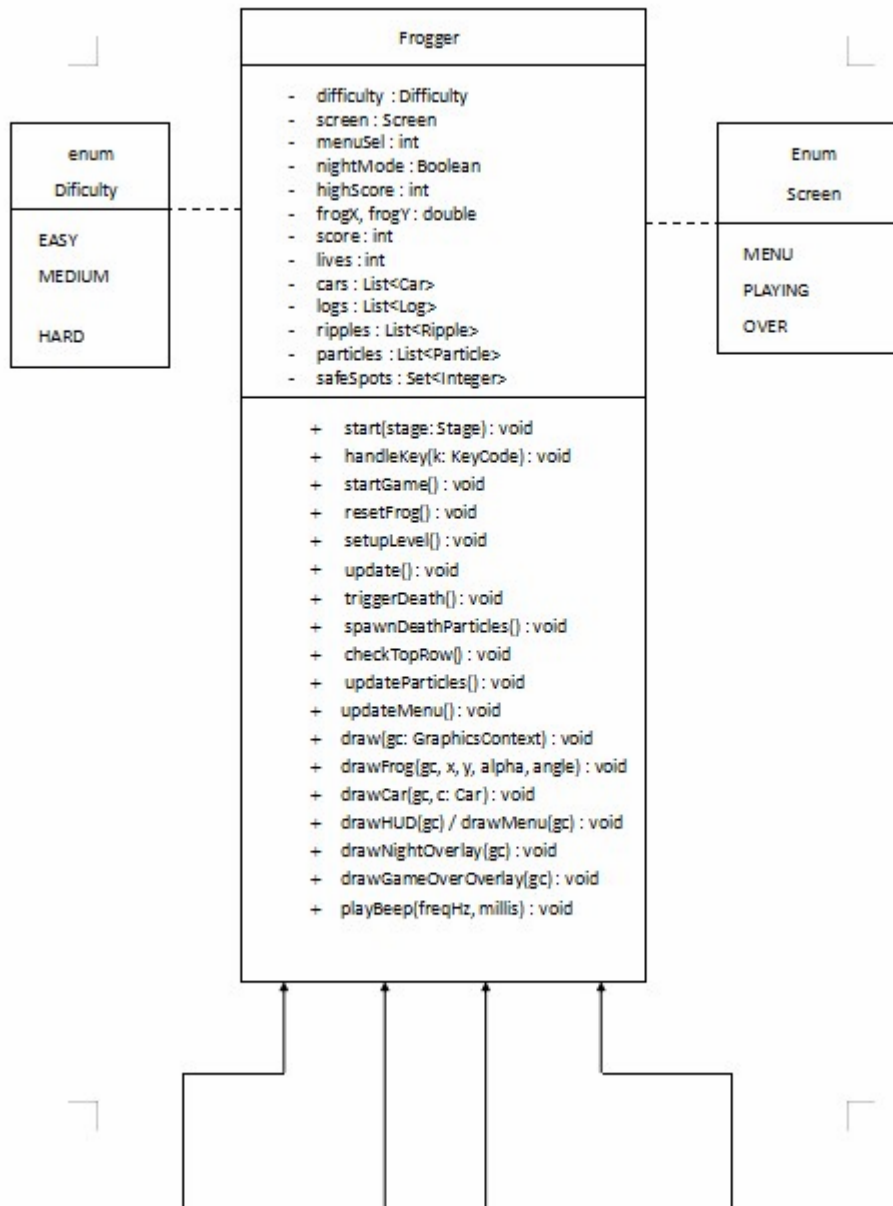
The playBeep() method generates sound effects using Java's built-in audio system without external files. It creates a short waveform signal to produce beep sounds for actions such as movement and collisions. This improves player feedback and enhances gameplay experience.

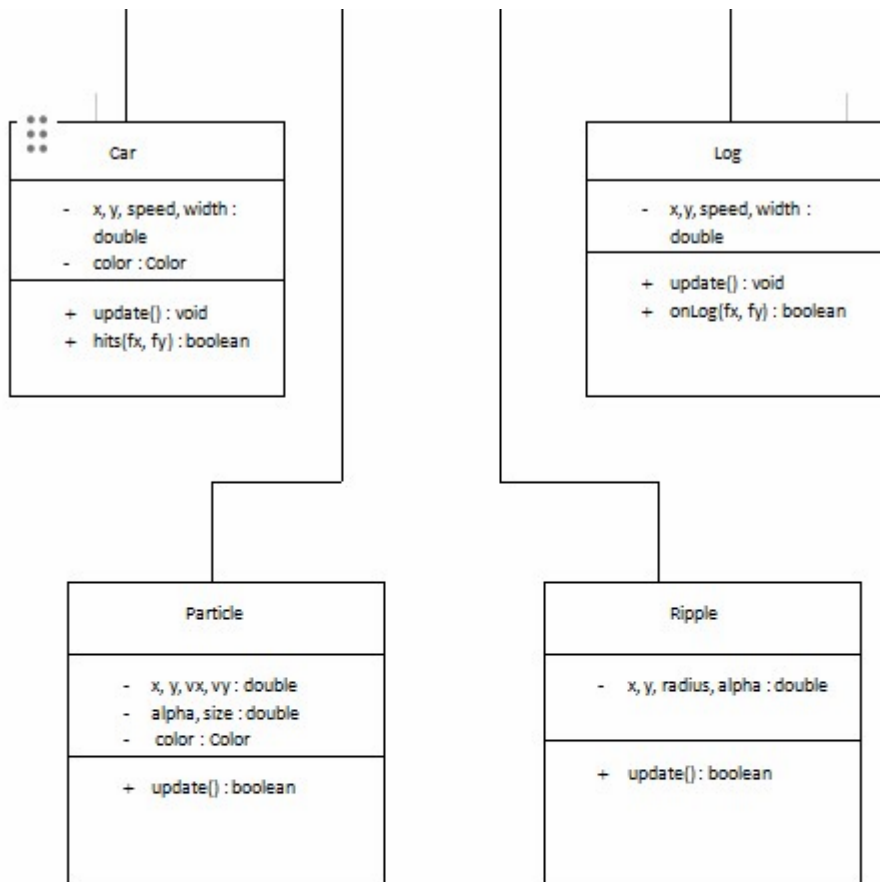
- **Trigger Death Method**

The triggerDeath() method handles what happens when the frog dies. It starts the death animation instead of instantly resetting the player. A screen shake effect is activated to enhance feedback, and particle effects are generated for visual impact. A sound effect is also played. After the animation finishes, the game reduces lives and either resets the frog or ends the game if no lives remain.

6.UML Diagram of Working

UML Diagram of code working is on next page.





Module	Key Methods	What It Does
Frogger (main)	start, update, draw, handleKey,	Owns all game state and runs the main

	setupLevel	loop
Car	update(), hits(fx, fy)	Moves on the road and detects frog collision
Log	update(), onLog(fx, fy)	Floats in the river and detects if frog is riding
AnimationTimer	handle(long now)	Drives the 60 fps loop, calling update then draw
GraphicsContext	fillRect, fillOval, fillText, strokeLine	Draws every shape and piece of text on screen
Scale transform	setX(), setY()	Stretches the canvas to fill any window size
Key listener	setOnKeyPressed	Reads keyboard input and routes it to handleKey

7. Team Contributions

The project was split into clear modules so each team member could focus on one area. The table below shows who built which part and what they learned from it.

Member	Part Assigned	What They Learned
Member 1	Game setup, constants, screens, difficulty system, game loop, scaling	JavaFX lifecycle, game states, scaling, animation loop
Member 2	Key controls, frog movement, menu control, level setup, start/reset game	Event handling, grid movement, inputbased control, difficulty balancing
Member 3	Update logic, collision system, death animation, particles, ripple effects, scoring	Game loops, collision detection, animation states, visual effects
Member	Part Assigned	What They Learned
Member 4	Drawing system, HUD, menu UI, overlays, night mode, sound	Canvas rendering, UI design, audio generation, rendering order

	system	
--	--------	--

8.. Key Learnings

JavaFX Basics

- The window hierarchy goes Stage, then Scene, then Group, then Canvas.
- Graphics Context is the only drawing tool for Canvas and must be used in the right order.
- AnimationTimer provides a game loop that runs approximately 60 FPS and automatically handles timing without manual thread management. Key events are caught on the Scene, not on individual shapes.

Object-Oriented Design

- Inner classes like Car and Log keep related data and behaviour in one place.
- Enum types replace hard-coded numbers and strings with readable names.
- Switch expressions give a short and clear way to handle multiple difficulty values.

Game Programming Ideas

- The game loop pattern of update then render is used in almost every real-time game.
- Axis-aligned bounding box collision is fast and works well for grid-based games.
- A sprite can be drawn entirely from basic shapes without needing image files.
- Object wrapping makes it look like there are infinite lanes of traffic.

Coordinates and Scaling

- Keeping all game logic in a fixed coordinate space and using a Scale transform lets the window resize freely without touching any game code.
- Property listeners on the Scene width and height automatically adjust the scale each time the user resizes the window.

9.How to Execute the Program

Steps to Run the Game

1. Install Java JDK and JavaFX SDK.
2. Open the project in IntelliJ IDEA or another Java IDE.
3. Add JavaFX libraries to the project build path.
4. Run the Frogger.java file.
5. Select a difficulty level and press Enter to start the game.

Conclusion

The Frogger project is a complete, playable arcade game built using only standard Java and JavaFX libraries. No external game frameworks were used. Every feature, including moving sprites, difficulty levels, and a scalable window system, was implemented from scratch. Developing this project provided practical experience with the JavaFX rendering pipeline, real-time game loops, event-driven input handling, and object-oriented programming design. Dividing the work into separate modules allowed each team member to contribute independently and efficiently without conflicts. The final result is a fully functional Frogger-style game with three difficulty levels, animated cars and logs, river riding mechanics, collision detection, a resizable window, a menu system, score tracking, and win/lose screens, all implemented in a single JavaFX application.